Lab1 刚体仿真（VCX）——实现说明

学号：2400012935　姓名：陈羿桥

仓库：https://github.com/CharlesChen397/PhySim_VCX

邮箱：charleschen397@gmail.com

2026 年 4 月 9 日

1　使用说明

- 构建与运行：`xmake build lab1`，`xmake run lab1`。
- 物理与约束：`RigidBodySystem.h/.cpp`。
- 场景与 UI：`CasePlayground.h/.cpp`。
- 三角网格：`PrimitiveMesh.cpp`（`BuildBodyMesh`）。

2　Task 1

平动为显式欧拉：重力与 `Force` 合成线加速度后积分 `V`、`X`。转动在体坐标下由 `InertiaBodyInv` 还原对角惯量，变到世界系得 $\mathbf{L} = \mathbf{I}\boldsymbol{\omega}$，欧拉方程 $\dot{\boldsymbol{\omega}} = \mathbf{I}^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{L})$。四元数用纯虚部 `wq` 与当前 `Q` 相乘，系数加 `0.5*dt` 项后归一化。`Fixed/Sleeping` 体跳过积分。

接触点冲量：$\Delta\mathbf{v} = \mathbf{J}/m$，$\Delta\boldsymbol{\omega} = \mathbf{I}^{-1}\mathbf{r} \times \mathbf{J}$。

```
void RigidBodySystem::integrateBodies(float dt) {
  for (auto & body : Bodies) {
    if (body.Fixed || body.Sleeping) continue;
    Eigen::Vector3f const gravityAccel(0.f, -Gravity, 0.f);
    Eigen::Vector3f const linearAccel = gravityAccel + body.Force * body.InvMass;
    body.V += linearAccel * dt;
    body.X += body.V * dt;
    Eigen::Matrix3f const R = body.Q.normalized().toRotationMatrix();
    Eigen::Matrix3f Ibody = Eigen::Matrix3f::Zero();
    for (int i = 0; i < 3; ++i) {
      float const invIi = body.InertiaBodyInv(i, i);
      if (std::abs(invIi) > 1e-12f) Ibody(i, i) = 1.f / invIi;
    }
    Eigen::Matrix3f const Iworld = R * Ibody * R.transpose();
    Eigen::Vector3f const L = Iworld * body.W;
    Eigen::Vector3f const tauNet = body.Torque - body.W.cross(L);
    Eigen::Vector3f const angularAccel = body.GetWorldInvInertia() * tauNet;
    body.W += angularAccel * dt;
    Eigen::Quaternionf const wq(0.f, body.W.x(), body.W.y(), body.W.z());
```

```
    body.Q.coeffs() += (0.5f * dt * (wq * body.Q).coeffs());
    body.Q.normalize();
  }
}
```

```
void RigidBody::ApplyImpulse(Eigen::Vector3f const & worldPoint,
                             Eigen::Vector3f const & impulse) {
  if (Fixed) return;
  Sleeping = false;
  SleepTimer = 0.f;
  V += impulse * InvMass;
  Eigen::Vector3f const r = worldPoint - X;
  W += GetWorldInvInertia() * r.cross(impulse);
}
```

3 Task 2

`createGeometry` 按 `ShapeType` 构造 `fcl::Box`、`fcl::Sphere`、`fcl::Cylinder`。**圆柱与 FCL 约定对齐：** FCL 的 `Cylinder` 对称轴为局部 $+z$，而本工程惯量与 `PrimitiveMesh::buildCylinder` 以局部 $+y$ 为轴；`createTransform` 对圆柱在体姿态右乘固定旋转（绕 x 旋转 $-90°$），使 FCL 局部与体轴一致，避免碰撞体与网格错位。

`collidePair` 用 `fcl::collide`，`maxContact` 控制返回接触数；`createTransform(x, q, shape)` 按形状决定是否附加圆柱对齐旋转。

`detectCollisions` 在写入接触列表前经 `appendContactsForPair`：若同一对刚体返回的多个接触法向彼此几乎平行（点积 ≥ 0.985），则合并为**单点**——法向为法向和的单位化，接触点为位置平均，穿透取最大。用于抑制面–面接触时多点顺序冲量造成的虚假角速度；法向仍按 $\overrightarrow{AB}$ 与法向一致性统一为从 `i` 指向 `j`。恢复系数取两体最小值。

速度级约束用多轮 Gauss–Seidel：相对法向速度 v_n，Baumgarte 把穿透误差压进 `biasVel`；有效质量 `effMass` 含平动与转动耦合。法向冲量 `jn` 非负。切向用 Coulomb 锥，`frictionNormal` 用 `jn` 减去由 `biasVel` 贡献的部分，避免深穿透时摩擦过大。

```
fcl::CollisionObject<float> objA(geomA, createTransform(xa, qa, a.Shape));
fcl::CollisionObject<float> objB(geomB, createTransform(xb, qb, b.Shape));
```

```
Eigen::Vector3f const ab = Bodies[j].X - Bodies[i].X;
if (ab.dot(n) < 0.f) n = -n;
contact.Restitution = std::min(Bodies[i].Restitution, Bodies[j].Restitution);
```

```
float jn = (-(1.f + restitution) * vn + biasVel) / effMass;
jn = std::max(0.f, jn);
float const frictionNormal = std::max(0.f, jn - biasImpulsePart);
jt = std::clamp(jt, -maxFrictionImpulse, maxFrictionImpulse);
```

4 Task 3

单步仿真顺序如下：若开启重叠松弛，先在积分前按穿透深度把重叠体沿法向推开一小步；保存上一帧位姿供连续碰撞检测；对全体刚体做显式积分；若开启 CCD，在高速接近且上一帧与当前帧均未报告穿透时，对位姿插值求首次接触时刻并回拨；再对所有刚体对做 FCL 碰撞检测（经接触合并）生成接触；解约束时若走 Schur 路径，则接触与关节乘子在同一线性系统里一次解出，否则先顺序解接触再顺序解关节；随后做位置穿透修正与静置接触稳定；最后清空本步外力累加器。

重叠松弛按接触深度与两体逆质量比例分配位移；位置修正对单次深度与 Baumgarte 相关项设有上限（如 `MaxDepthPerPositionPass`、`BiasPenetrationCap`），减轻深穿透与堆叠初始化的数值冲击。

```
void RigidBodySystem::Step(float dt) {
  if (dt <= 0.f || Bodies.empty()) return;
  if (RelaxOverlapsBeforeIntegrate) relaxOverlappingPositions();
  _prevStates.resize(Bodies.size());
  for (std::size_t i = 0; i < Bodies.size(); ++i)
    _prevStates[i] = BodyState { .X = Bodies[i].X, .Q = Bodies[i].Q };
  integrateBodies(dt);
  if (EnableCCD) continuousCollisionPass(dt);
  detectCollisions();
  if (EnableSchurComplement) solveContactsSchur(dt);
  else { solveContactsSequential(dt); solveJointsSequential(dt); }
  solvePositionCorrection();
  stabilizeRestingContacts(dt);
  for (auto & body : Bodies) body.ClearAccumulators();
}
```

```
Eigen::Vector3f const corr = (relaxFactor * depth / invMassSum) * c.Normal;
if (!a.Fixed) a.X -= a.InvMass * corr;
if (!b.Fixed) b.X += b.InvMass * corr;
```

5 Bonus B1

五个等尺寸立方体沿 $+x$ 排成接触链：静止四块中心在 $0, 2h, 4h, 6h$（h 为半长），首块（撞击块）中心在 $-2h - \texttt{strikerGap}$，与第一块静止体之间留约 $0.88\,\mathrm{m}$ 间隙，初速约 $(5, 0, 0)$，便于观察加速与碰撞传递过程。恢复系数约 0.992、摩擦为 0，并提高速度迭代次数、略降时间尺度与子步，减轻串行冲量下的能量损失与顺序误差。面–面类预设中可对平行法向接触做合并（见 Task 2），减轻虚假转动。仿真由用户通过 **Start** 启动，无自动侧向冲量。

6 Bonus B2

地面上的多层立方体：零恢复、高摩擦；依赖子步数与位置迭代抑制穿透与抖动。

7 Bonus B3

FCL 使用球、柱碰撞体；惯量按球、柱解析公式写入体对角逆（圆柱轴向与 y 轴一致）。渲染用 `BuildBodyMesh` 为球、柱生成三角网格。圆柱网格高度沿 $\pm y$；碰撞侧通过绕 x 轴 -90° 的固定旋转

与 FCL 圆柱局部 z 轴对齐。场景中含立方体、球、柱与地面对象，柱体可给初角速度演示滚动。

```
MeshData BuildBodyMesh(RigidBody const & body) {
  switch (body.Shape) {
  case ShapeType::Box: return buildBox(body);
  case ShapeType::Sphere: return buildSphere(body);
  case ShapeType::Cylinder: return buildCylinder(body);
  }
  return buildBox(body);
}
```

8 Bonus B4

Schur 路径下，每个接触与每个关节轴对应一行约束：行内有世界系法向、力臂与右端项（含恢复系数与法向速度、Baumgarte 穿透修正；接近静止时削弱穿透项）。全局矩阵 $\mathbf{A}$ 的元素由两体逆质量与逆惯量对行对行的耦合构成；对角加小正则后用 LDLT 解乘子向量。单边行做 $\lambda \geq 0$ 投影，并对法向乘子按质量、重力与时间步设上限，避免单步冲量过大。

```
A += 2e-4f * Eigen::MatrixXf::Identity(m, m);
Eigen::VectorXf lambda = A.ldlt().solve(-b);
for (int i = 0; i < m; ++i) {
  float li = lambda(i);
  if (rows[i].Unilateral) {
    li = std::max(0.f, li);
    float capRest = mDyn * gEff * std::max(dt, 1e-5f) * SchurImpulseGravityMult;
    // ... 碰撞态放大 cap ...
    li = std::min(li, cap);
  }
  Eigen::Vector3f const impulse = li * rows[i].N;
  Bodies[rows[i].A].ApplyImpulse(rows[i].Point, -impulse);
  Bodies[rows[i].B].ApplyImpulse(rows[i].Point, impulse);
}
```

9 Bonus B5

固定根立方体与五节细长连杆**端面对端面**沿 x 排成一条直线（质心共线、y, z 相同，相邻面间留小间隙）；每对相邻刚体之间仅施加**一个**点对点约束，世界锚点取**间隙中面**上截面中心 $(x_{\text{mid}}, y_c, z_c)$。双锚点顺序冲量会在两约束间交替抢误差导致振荡，故采用单点球铰配合几何对齐。关节速度级 Baumgarte 系数由成员 `JointBaumgarteBeta` 控制（本场景约 0.055），默认 0.2；并略增速度迭代次数。

数据结构仍用局部锚点；顺序求解时构造 3×3 有效质量矩阵，$\texttt{rhs} = -(\mathbf{v}_{\text{rel}} + \beta\,\mathbf{e}/\Delta t)$，LDLT 解冲量。Schur 路径下关节三轴与接触同行联立。

```
float const beta = JointBaumgarteBeta;
Eigen::Matrix3f K = (a.InvMass + b.InvMass) * Eigen::Matrix3f::Identity();
K -= skew(ra) * invIA * skew(ra);
K -= skew(rb) * invIB * skew(rb);
Eigen::Vector3f const rhs = -(rv + beta * err / std::max(dt, 1e-6f));
Eigen::Vector3f const impulse = K.ldlt().solve(rhs);
```

```
a.ApplyImpulse(pa, -impulse);
b.ApplyImpulse(pb, impulse);
```

10 Bonus B6

CCD 仅在两体均非固定、相对速度非零时考虑。跳过条件改为：$\|\mathbf{v}_i - \mathbf{v}_j\|\Delta t \le k\,\eta$，其中 η 为两物体沿相对速度单位方向上的**投影特征尺度**（球为直径；盒为 OBB 三半轴在该方向投影的 L_1 型宽度），$k \approx 0.22$，避免薄板沿弹道方向极薄时仍误判为可跳过而穿模。

若上一帧与当前帧均未穿透，则在 $t \in [0, 1]$ 上对位置线性插值、姿态 `slerp`，约 22 次二分求最早碰撞；若仍未命中且位移足够大，再对 t 做 32 步均匀扫描作后备。碰撞查询使用 `maxContact=4`。回拨时刻取 `hi` 减去与弧长相关的微小裕量。B6 场景中薄板与高速小球并设子步数为 4。

11 用户输入与冲量

左侧面板：下拉框切换场景；**Reset Preset** 重置当前场景（含 `JointBaumgarteBeta` 等求解器参数默认值）；**Start/Stop Simulation** 启停仿真。**Selected Body** 选择刚体；**Impulse Strength** 与线速度（Vx/Vy/Vz）、角速度（Wx/Wy/Wz）六滑条置于上方，便于调节运动状态；六个 **Impulse ±X/Y/Z** 按钮沿世界轴对选中体质心施一次冲量；**Gravity** 调节重力。

鼠标：在画面获得焦点且已选中非固定刚体时，按住拖动会通过相机位移换算成作用在该体质心上的冲量（力度与拖动幅度相关）。

键盘：在画面获得焦点时，**J/L** 沿 $\pm X$，**I/K** 沿 $\mp Z$，**U/O** 沿 $\pm Y$ 各施一次冲量，大小由 **Impulse Strength** 决定。